

MEMORYLACE: Memory Lifecycle-Aware Consolidation and Evidence Retrieval

Meriem Yacoubi^{*†}, Pia Schmidt[†], Nenad Petrovic^{*}, Ahmed Frikha[‡], Martin Kirchhoff[†], Alois Knoll^{*}

^{*} Chair of Robotics, Artificial Intelligence and Real-Time Systems, Technical University of Munich

[†] inovex GmbH

[‡] Cerebras Systems Inc.

Email: {meriem.yacoubi, nenad.petrovic, k}@tum.de,
{pia.schmidt, martin.kirchhoff}@inovex.de, ahmed.frikha@cerebras.net

Abstract—Long-term LLM agents must preserve information across interactions while distinguishing repeated evidence, historical states, updates, and unresolved contradictions. Existing textual memory systems retrieve semantically relevant memories efficiently but often leave these relationships implicit, whereas richer structured approaches model them through global graphs, hierarchical abstractions, or reflection at greater complexity. We introduce MEMORYLACE (MEMLACE), a lightweight memory framework that explicitly models the lifecycle of textual evidence through sparse merge, supersession, and contradiction relations while preserving atomic natural-language memories and their provenance. Rather than retrieving memories independently, MEMLACE reconstructs relation-aware evidence units that expose current, historical, supporting, and conflicting evidence for downstream reasoning. Across BEAM and StructMemEval, using open-weight and proprietary LLM backbones, MEMLACE achieves the highest overall performance in same-backbone comparisons while reducing end-to-end runtime on BEAM by 66.6% relative to Hindsight, the strongest reported reflective-memory baseline. Ablation studies identify lifecycle expansion and temporal awareness as the principal contributors to these gains. Together, the results demonstrate that explicitly modeling the local lifecycle of textual evidence is sufficient to substantially improve long-term memory reasoning without requiring comprehensive knowledge graphs or global reflection.

Index Terms—Long-term memory, LLM agents, evidence lifecycle, memory consolidation, contradiction handling

I. INTRODUCTION

Large Language Models (LLMs) increasingly support agents that interact with users across tasks and sessions. Beyond a single context window, these agents require memory mechanisms that preserve preferences, plans, corrections, constraints, task progress, and previously established facts [1]. External memory can ground generation [2], support persistent behavior [3], and extend long-term interaction [4]. However, reliable memory requires more than retrieving relevant text: new evidence may repeat, replace, or contradict stored information. Existing approaches expose a practical trade-off. Compact textual systems offer efficient storage and retrieval [5], [17], but often leave relationships between memories implicit. Structured, hierarchical, and reflective architectures represent these relationships explicitly [9]–[11], [24], but may require broader graph construction, abstraction, or reflection. This motivates a focused question: how can an agent preserve

changes in textual evidence and retrieve dependent memories jointly without constructing a comprehensive global structure?

We introduce MEMORYLACE (MEMLACE), a lightweight middle ground between flat textual and comprehensive structured memory. MEMLACE preserves atomic natural-language memories and their provenance while connecting related entries through sparse local relations for merge, supersession, and contradiction. During writing, bounded candidate comparison and local batch evolution establish and repair these relations. During retrieval, active memories provide initial anchors, lifecycle expansion recovers connected historical or conflicting evidence, and relation-aware evidence units are reranked and packed under the context budget. The answer model therefore receives current, historical, supporting, and conflicting evidence in an interpretable structure rather than as independent snippets.

Our contributions are threefold. First, we introduce an explicit lifecycle representation and consolidation framework that distinguishes repeated evidence, ordered state changes, and unresolved contradictions while preserving atomic natural-language memories and their provenance. Second, we propose a relation-aware retrieval mechanism that reconstructs lifecycle-aware evidence units by expanding active memories through lifecycle relations, enabling the language model to jointly interpret current, historical, supporting, and conflicting evidence rather than isolated memory snippets. Third, through comprehensive evaluation on BEAM and StructMemEval using both open-weight and proprietary LLM backbones, together with ablation studies, we demonstrate that explicitly modeling only local evidence lifecycle structure is sufficient to substantially improve long-term memory reasoning, achieving the strongest overall performance in same-backbone comparisons without requiring a comprehensive knowledge graph or global reflection stage.

II. RELATED WORK

Recent surveys characterize agent memory through how information is represented, used, evolved, and retrieved over time [1]. Within textual memory, Mem0 extracts salient conversational information, reconciles it with stored entries, and retrieves the resulting memory objects across sessions [17],

while SimpleMem emphasizes semantic compression, recursive consolidation, and query-aware retrieval [5]. SeCom studies memory-unit granularity and constructs compressed memory segments organized around coherent topics [6], and MemInsight augments stored interactions to improve semantic representation and retrieval [7]. EMem represents dialogue as structured event-level propositions with normalized entities, temporal cues, and source attribution, optionally connecting them through a lightweight heterogeneous graph [8]. These systems show that compact textual or event-level memories can provide strong long-horizon recall, but generally do not represent repetition, supersession, and unresolved contradiction as explicit lifecycle relations.

A-MEM, Zep, Theanine, TiMem, and HiMem provide closely related approaches to evolving and temporally organized memory. A-MEM constructs structured notes and allows new evidence to trigger the evolution of linked representations [9]. Zep uses a temporally aware knowledge graph [24], Theanine links memories through temporal and causal timelines [12], and TiMem and HiMem consolidate observations into hierarchical episodic and semantic representations [10], [11]. These systems demonstrate the value of temporal continuity, but their representations model broader semantic, causal, or hierarchical structure. MEMLACE instead creates only local merge, supersession, and contradiction relations when they alter the interpretation of stored evidence.

Reflective Memory Management, Hindsight, and MemOS address memory evolution through richer maintenance mechanisms. Reflective Memory Management maintains both prospective and retrospective user representations [16], while Hindsight separates retained observations, recalled information, and reflective interpretations across distinct memory networks [15]. MemOS treats memory as a governed system resource, supporting provenance, versioning, composition, migration, and evolution across heterogeneous memory forms [18]. These approaches reinforce the importance of preserving evidence while allowing its interpretation and operational status to evolve. Rather than introducing dedicated reflection modules or a memory operating layer, MEMLACE limits maintenance to bounded candidate comparison, sparse lifecycle relations, and relation-aware retrieval.

Benchmarks for long-term memory have moved beyond simple conversational recall. LoCoMo and LongMemEval probe multi-session question answering, temporal reasoning, knowledge updates, and abstention [20], [21], while MemBench and Evo-Memory add efficiency, capacity, and continuous memory evolution as evaluation axes [22], [23]. We evaluate on *BEAM* [13] and *StructMemEval* [14], which test complementary abilities: *BEAM* measures long-context conversational memory across ten capabilities, whereas *StructMemEval* isolates whether accumulated evidence can be organized to support structured reasoning. Together they cover both the interpretation of evolving evidence and its structured use, the two settings our method targets.

III. MEMORYLACE ARCHITECTURE

MEMLACE treats long-term memory as evolving evidence rather than a flat collection of summaries. The architecture comprises a memory representation and two stages. The representation models atomic textual memories with temporal metadata, provenance, activity state, and sparse lifecycle relations. The first stage, *memory construction and consolidation*, converts dialogue into standalone memories, stores them, and determines how new evidence relates to existing memories. The second stage, *evidence retrieval and context construction*, retrieves active anchors, i.e., active memories selected as relevant to the query, expands their local relations, and organizes connected entries before answer generation. This design enables the system to retrieve relevant evidence while preserving its evolution over time, without requiring a comprehensive, globally normalized knowledge graph.

A. Memory Representation

Each stored memory is represented as

$$m_i = (x_i, \tau_i, z_i, p_i, a_i, R_i),$$

where x_i is a standalone natural-language restatement of one atomic piece of explicit dialogue evidence. The restatement preserves the original meaning and relevant details while making the memory independently understandable outside the original conversation context. The field τ_i records evidence-grounded temporal metadata, such as an explicitly stated event time or validity period, while z_i contains lightweight metadata such as entities, keywords, topics, and locations. The provenance field p_i records the identifiers of the supporting dialogue turns, thereby preserving the order and origin of the evidence; a_i indicates whether the memory is active; and R_i denotes the set of lifecycle-relation references associated with the memory, including zero or more merge, supersession, and contradiction relations. Here, the lifecycle of a memory is how its status evolves as new evidence arrives: whether it remains active and how it relates to repeated, superseded, superseding, or conflicting memories. Supersession is represented in both directions: newer memories point to the entries they replace, and older memories point to their successors.

The complete memory store forms a sparse relation graph $G = (M, E)$, where M is the set of textual memory nodes and E comprises directed supersession edges and symmetric merge and contradiction links. Unlike a general knowledge graph, which may encode entity, taxonomic, causal, or semantic relations, MEMLACE stores only relations needed to interpret stored evidence. Provenance identifies a memory’s source, whereas lifecycle relations describe its connections to other memories. This preserves the flexibility of natural-language memory while making its evolution inspectable.

B. Stage I: Memory Construction and Consolidation

Figure 1 illustrates Stage I. The process follows the lightweight sliding-window memory construction of SimpleMem [5]: overlapping dialogue windows are converted into standalone atomic memories, and duplicate entries introduced

by the overlap are consolidated before lifecycle processing. MEMLACE extends this process with lifecycle-aware consolidation for evidence that evolves over time.

1) *Memory Construction*: For a dialogue window D_t , the memory builder produces

$$B(D_t) \rightarrow \{m_1, m_2, \dots, m_k\}.$$

Each atomic entry captures a single explicit fact, preference, plan, correction, decision, event, or state, together with source-turn provenance that links it to the supporting dialogue evidence. Names, dates, quantities, negations, and speaker attribution are preserved as stated, and no unstated information is inferred; mentioning a trip to Berlin, for instance, does not imply residence there. This atomic granularity is important because an update or contradiction typically affects only a single claim within a broader dialogue segment, allowing consolidation to act on precisely the affected memory. Since overlapping windows may extract the same claim from the same turns more than once, such duplicate extractions are collapsed and their provenance combined, whereas repetitions from distinct turns remain separate for possible merge linking.

2) *Candidate and Relation Selection*: Memories are consolidated in chronological order so that later evidence cannot be treated as preceding the state it modifies. For each memory m_{new} , MEMLACE first retrieves related memories through semantic similarity and same-entity matching, then expands this set with their one-hop lifecycle neighbors to form the candidate set $C(m_{\text{new}})$. These neighbors are memories connected through merge, supersession, or contradiction relations. A cross-encoder reranks the candidates, preserving lifecycle-linked memories and prioritizing the highest-ranked remaining candidates. Reranking determines only which memories are considered for comparison; it does not determine their relation to m_{new} .

The consolidation module subsequently selects exactly one strategy r and, when applicable, one target memory. Restricting each step to a single relation–target pair lets the model focus on one decision at a time, which makes the strategy decision more reliable than jointly assigning relations to several candidates:

$$\mathcal{R} = \{\text{merge}, \text{update}, \text{contradict}, \text{no_action}\},$$

$$g(m_{\text{new}}, C(m_{\text{new}})) \rightarrow (r, c^*), \quad r \in \mathcal{R}.$$

Here, g denotes the consolidation decision function, and $c^* \in C(m_{\text{new}})$ denotes the selected target memory, or null when no target is required. *Merge* is applied to duplicate evidence about the same fact, *update* to a time-ordered change of the same state, *contradict* to incompatible claims with the same scope, and *no action* to independent memories. Because retrieval identifies only potentially related candidates, the consolidation model assesses their subject, attribute, claim, and temporal scope before assigning a relation, preventing topical similarity alone from altering a memory’s lifecycle state. Relations to additional memories are recovered by the subsequent local batch step rather than forced into this single decision.

3) *Lifecycle State Update*: Under the default configuration, lifecycle decisions modify memory activity, i.e., whether a memory belongs to the active set A , and connectivity, i.e., its lifecycle relations to other memories, without deleting source evidence. Let

$$A = \{m_i \in M \mid a_i = 1\}$$

denote the active subset of the complete memory store M . When m_n updates m_o , the older entry becomes inactive and the newer entry becomes the active chain head:

$$\begin{aligned} a_o &\leftarrow 0, & a_n &\leftarrow 1, \\ m_n &\xrightarrow{\text{supersedes}} m_o, & m_o &\xrightarrow{\text{superseded_by}} m_n. \end{aligned}$$

Contradictory memories remain active because the available evidence does not resolve their conflict, and they are linked symmetrically. Memories assigned no action remain independently retrievable. For merge, we explore two consolidation policies. Under the *default linked-merge* policy, memories assigned the merge relation retain their original text and provenance while receiving bidirectional merge references, so that each original piece of evidence remains separately retrievable. In contrast, the *compact-merge* policy replaces the merged entries with one synthesized active memory, producing a smaller memory store at the cost of discarding the original individual entries. Unless otherwise stated, MEMLACE uses the default linked-merge policy. Independently, superseded states are excluded from ordinary active retrieval while historical, merge-linked, and conflicting evidence remains stored.

4) *Local Batch Memory Evolution*: Each memory is consolidated through a single lifecycle decision, keeping every per-memory step bounded and inspectable while reducing sensitivity to decision order. After a new group of memories has been added, MEMLACE complements this with a bounded local evolution step that considers the group jointly. It reviews the newly added memories together with their Stage I consolidation candidates and establishes any further update, contradiction, or merge links that become apparent only when the group is viewed together, such as a memory relating to more than one earlier entry. This step acts only on lifecycle structure, updating relation links and corresponding activity states without rewriting memory text or adding factual content. Coordinating single-decision consolidation with group-level evolution therefore repairs additional lifecycle relations while preserving the bounded, local decision process.

C. Stage II: Evidence Retrieval and Context Construction

Figure 2 summarizes the query-time pipeline.

1) *Hybrid Active-Memory Retrieval*: At query time, we use semantic similarity, keyword match, structured template relevance, and topic relevance to produce complementary ranked lists over active memories. The resulting rankings are combined using Reciprocal Rank Fusion (RRF) [25], which does not require their raw scores to be directly comparable:

$$s_{\text{RRF}}(m) = \sum_{c \in \mathcal{C}} \frac{1}{k + \text{rank}_c(m)},$$

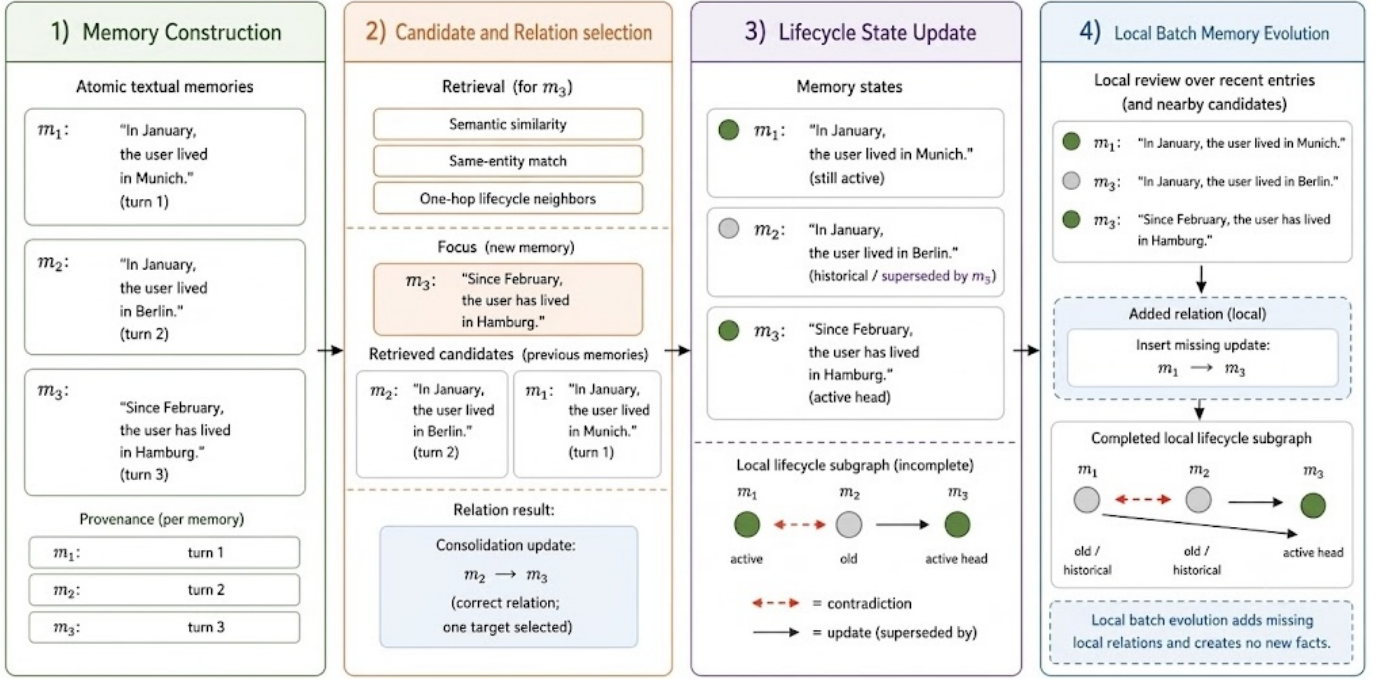


Fig. 1. Memory construction and consolidation in MEMLACE. Atomic provenance-bearing memories undergo bounded relation selection, lifecycle-state updates, and local batch repair of missed relations.

where $\mathcal{C}$ denotes the retrieval channels and k is a rank-smoothing constant. Restricting the initial search to active memories reduces competition from superseded states. Inactive memories remain stored and can be recovered via retrieved anchors' lifecycle relations when historical evidence is required. Formally, hybrid retrieval returns the set of active anchors

$$A_q = R_{\text{hybrid}}(q, A),$$

where A is the active-memory set and R_{hybrid} fuses the retrieval channels above through RRF.

2) *Lifecycle Expansion and Evidence Units*: Hybrid retrieval returns relevant active anchors A_q , which alone may not contain all evidence required to interpret how the corresponding information evolved over time. MEMLACE therefore expands each anchor through bounded lifecycle relations,

$$E_q = A_q \cup L(A_q),$$

where $L(A_q)$ follows supersession lineages to recover prior states and one-hop contradiction relations to surface directly conflicting evidence. The expansion remains local rather than traversing the full memory graph, limiting additional context introduced for each query while still recovering the historical and conflicting evidence that a query may depend on.

The resulting memories are grouped into evidence units. A unit can represent a standalone memory or a connected component formed by supersession, contradiction, and merge links among memories in the retrieved pool. Supersession edges arrange each component from earlier to newer states so that both historical evidence and the active state remain visible.

Unresolved contradictory claims are retained together rather than prematurely resolving one in favor of another. Grouping before reranking allows memories whose interpretation depends on their relations to be evaluated as a coherent unit instead of competing as independent snippets.

3) *Chain-Aware Reranking and Context Packing*: A reranker scores the resulting lifecycle-aware evidence units against the user query, after which the highest-scoring units are selected for answer generation. Because evidence units are scored as whole groups, relation-connected memories are evaluated jointly during reranking rather than competing as independent snippets. The highest-ranked units are then selected subject to the context budget. Update chains preserve their older-to-newer order, and complete evidence units are retained whenever the budget permits. When a unit would exceed the remaining budget, it is included only in part, keeping its higher-priority memories and dropping the rest.

4) *Evidence-Grounded Answer Generation*: During answer generation, the packed context is presented in lifecycle order, distinguishing current, superseded, and conflicting memories to the LLM. The model can therefore prioritize active evidence for current-state questions, recover earlier states for historical or sequential questions, and retain competing claims when contradictions are unresolved, receiving relation-connected evidence with its temporal structure preserved rather than inferring relations from unordered snippets.

IV. EXPERIMENTAL SETUP

We evaluate MEMLACE on two long-term memory benchmarks. *BEAM* [13] is the main benchmark for conversa-

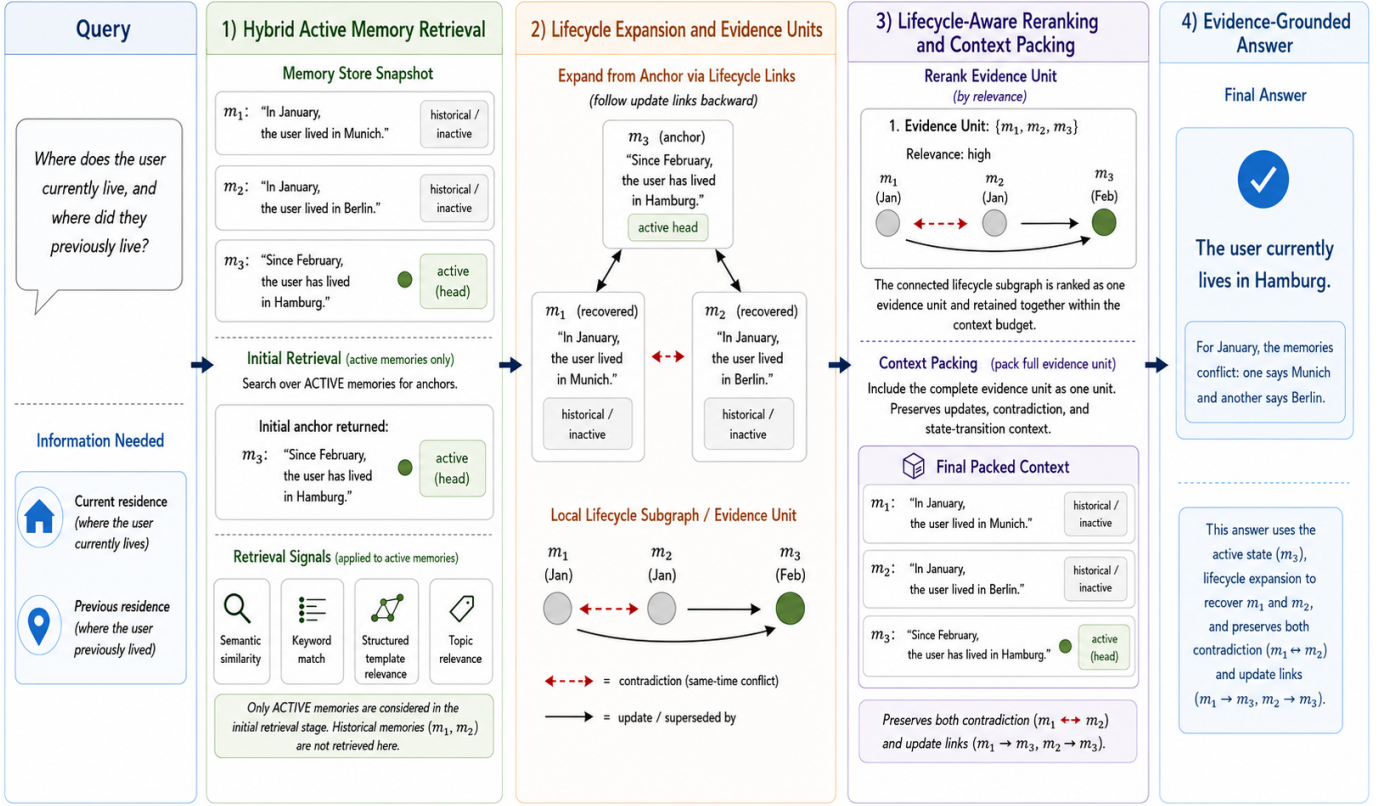


Fig. 2. Evidence retrieval and context construction in MEMLACE. Active anchors are expanded through lifecycle relations, grouped into evidence units, reranked, and packed for evidence-grounded generation.

tional long-term memory, while *StructMemEval* [14] assesses whether evidence can support structured reasoning.

A. Benchmarks

BEAM evaluates ten different memory capabilities over long conversational histories [13]. We use its 100K-token subset, which contains 20 conversations and 400 questions. *StructMemEval* evaluates state tracking, tree-based reasoning, count-based reasoning, and recommendation over persistently stored information [14]. We use its 51-scenario main set, comprising the longest cases in each task family.

B. Experimental Configuration

For a fair comparison, we align MEMLACE with the model and evaluation settings of the corresponding baselines. On *BEAM*, the evaluated backbones are Qwen3.5-4B and Qwen3.5-9B (both FP8) [26]. On *StructMemEval*, MEMLACE and Mem-Agent are evaluated with GPT-5.5 [27]. For each benchmark, the backbone is used for memory construction, consolidation, batch evolution, query analysis, and answer generation. Generated answers are scored by GPT-4o mini [28] as the fixed LLM judge following each benchmark’s evaluation protocol. Reasoning mode was disabled for all Qwen3.5 runs, and the generation temperature was set to 0.0.

Memory construction uses 20-turn windows with a 2-turn overlap. Consolidation retrieves 12 semantic and 8 entity

candidates under a total candidate budget of 15, reranks the five highest-scoring candidates, and reviews at most 24 entries during batch evolution. Query-time retrieval uses active-only anchors and retrieves up to 40 semantic, 30 keyword, 30 structured, and 20 topic candidates, which are fused using RRF with $k = 60$. Supersession-lineage expansion is bounded at 80 nodes, contradiction expansion at four memories, and the final context at 64 memories. Semantic retrieval uses Qwen/Qwen3-Embedding-0.6B [30], while candidate and evidence-unit reranking use cross-encoder/ms-marco-MiniLM-L6-v2 [29].

V. RESULTS AND ANALYSIS

We evaluate MEMLACE on the *BEAM* 100K subset and the *StructMemEval* main set. On *BEAM*, we compare against LIGHT [13], reported by the benchmark as its highest-performing memory architecture, Hindsight [15], which uses a retain-recall-reflect pipeline with global reflection, and SimpleMem [5], a compact textual-memory system that shares our standalone construction scheme, isolating the contribution of the lifecycle representation and retrieval. We use Qwen3.5 models as backbones for our *BEAM* evaluations. On *StructMemEval*, we compare against Mem-Agent [19], a markdown-based memory agent, under the same GPT-5.5 backbone.

A. BEAM Category-Level Comparison

Table I reports our results across the ten *BEAM* memory capabilities. The column abbreviations denote abstention (Abst.),

TABLE I
CATEGORY-LEVEL RESULTS ON THE *BEAM* 100K SUBSET (IN %)

Method	Backbone	Abst.	Contr.	Event	Info.	Instr.	Update	Multi.	Pref.	Summ.	Temp.	Overall
SimpleMem	Qwen3.5-4B	38.8	7.2	21.9	56.0	29.4	31.9	26.9	53.1	7.7	32.5	30.5
LIGHT	Qwen3.5-4B	62.5	13.8	2.6	57.3	45.0	35.6	38.7	63.3	29.8	7.5	35.6
Hindsight	Qwen3.5-4B	50.0	6.6	18.7	50.0	59.4	30.0	40.3	85.0	33.4	28.1	40.1
MEMLACE	Qwen3.5-4B	88.8	36.6	29.3	52.9	61.9	25.0	33.8	81.3	27.8	16.9	45.4
SimpleMem	Qwen3.5-9B	37.5	10.0	21.2	51.2	30.0	36.9	33.7	48.8	7.8	30.0	30.7
LIGHT	Qwen3.5-9B	75.0	13.8	2.3	67.7	51.2	52.5	38.9	71.9	36.0	11.9	42.1
Hindsight	Qwen3.5-9B	63.7	40.9	20.8	61.1	68.1	51.2	50.3	71.2	40.4	35.0	50.3
MEMLACE	Qwen3.5-9B	71.3	40.3	25.6	55.9	72.5	51.9	55.3	84.4	35.5	26.3	51.9

contradiction resolution (Contr.), event ordering (Event), information extraction (Info.), instruction following (Instr.), knowledge update (Update), multi-hop reasoning (Multi.), preference following (Pref.), summarization (Summ.), and temporal reasoning (Temp.).

We find that MEMLACE substantially outperforms all baselines on the overall average at both model sizes. With Qwen3.5-4B as backbone, MEMLACE achieves the best results among the 4B systems in abstention, contradiction resolution, event ordering, and instruction following, indicating that the benefit of lifecycle structure already appears at smaller model scale without requiring a larger LLM backbone to compensate for disorganized context.

With the Qwen3.5-9B backbone, MEMLACE outperforms the strongest baseline, Hindsight, in six of ten categories. The gains are not spread uniformly: they concentrate in preference following, abstention, multi-hop reasoning, event ordering, and instruction following, precisely the categories in which the correct answer depends on considering several related memories together rather than in isolation. This is consistent with lifecycle-aware retrieval, which delivers the active state alongside the historical and supporting evidence connected to it. Knowledge update improves only marginally, which is expected: a single active-memory lookup already suffices and does not benefit from adding and grouping related entries.

Although MEMLACE achieves the highest 9B score, Hindsight retains an advantage of 8.7 percentage points in temporal reasoning, 5.2 percentage points in information extraction, and 4.9 percentage points in summarization. This pattern reflects the different design priorities of the two systems. MEMLACE deliberately uses atomic memories, sparse local relations, and bounded evidence packing to resolve evolving states efficiently, rather than reconstructing the complete dialogue or maintaining a globally reflective representation as Hindsight does. While this trade-off yields substantially higher efficiency, as shown in the next section, it is less suited to exhaustive summarization and explicit temporal-interval calculation.

1) *Runtime Comparison*: In this section, we compare the runtime needed by MEMLACE to that of the highest-performing baseline method, Hindsight, using Qwen3.5-9B as backbone. Table II reports the phase-level breakdown under the same evaluation settings, i.e., the *BEAM* 100K subset,

vLLM serving environment, parallel-processing configuration, and one NVIDIA A40 virtual GPU profile with 24 GB of allocated memory.

TABLE II
RUNTIME BREAKDOWN ON THE *BEAM* 100K SUBSET WITH QWEN3.5-9B.

Phase	MEMLACE	Hindsight	Speedup
Memory construction	6 h 06 min	18 h 27 min	3.02×
Answer generation	1 h 25 min	4 h 03 min	2.86×
Total	7 h 31 min	22 h 30 min	2.99×

MEMLACE reduces the total system runtime by 66.6%, a 2.99× speedup. The largest saving occurs in memory construction (3.02×), the phase in which MEMLACE employs bounded consolidation and local batch repair instead of the expensive global reflection stage used by Hindsight.

B. StructMemEval Results

The results on the *StructMemEval* benchmark are presented in Table III. We compare MEMLACE with Mem-Agent [19], the strongest among the memory systems evaluated on *StructMemEval* [14], using GPT-5.5 as the backbone LLM. A scenario counts as solved when at least half of its questions are judged correct, and the table reports the fraction of solved scenarios per subtask together with their macro-average.

TABLE III
StructMemEval RESULTS USING GPT-5.5 AS THE LLM (IN %).

Method	State	Tree	Count	Recsys	Overall
Mem-Agent	57.00	50.00	0.00	33.00	35.00
MEMLACE	100.00	100.00	0.00	8.33	52.08

MEMLACE outperforms Mem-Agent by more than 17 percentage points. MEMLACE solves all 14 state-tracking and all 10 tree-based scenarios, against 57.00% and 50.00% for Mem-Agent. The state-tracking result benefits directly from the memory representation we propose: supersession links preserve each state and active-head management resolves the current memory, thereby enabling correct answers to both current-state and historical questions. The tree-based result

TABLE IV
ABLATION STUDY RESULTS USING QWEN3.5-9B ON *BEAM* 100K (IN %), INCLUDING THE ABSOLUTE CHANGE FROM THE FULL MEMLACE CONFIGURATION (Δ). THE COMBINED VARIANT USES COMPACT MERGE AND DISABLES BOTH RERANKERS, BATCH EVOLUTION, TEMPORAL AWARENESS, AND LIFECYCLE EXPANSION.

Variant	Abst.	Contr.	Event	Info.	Instr.	Update	Multi.	Pref.	Summ.	Temp.	Overall	Δ
Full MEMLACE	71.3	40.3	25.6	55.9	72.5	51.9	55.3	84.4	35.5	26.3	51.9	–
Without both rerankers	70.0	39.1	23.4	57.9	63.8	56.3	50.8	80.6	37.3	27.5	50.7	-1.23
Compact merge	71.3	44.1	24.7	54.8	71.3	53.8	43.0	83.8	34.4	23.8	50.5	-1.43
Without batch evolution	72.5	36.9	24.1	56.5	60.6	55.0	48.2	83.3	38.1	26.9	50.2	-1.68
Without temporal awareness	72.5	26.6	23.8	54.4	67.5	48.8	46.9	75.6	37.0	16.3	46.9	-4.97
Without lifecycle expansion	73.8	40.3	23.4	47.8	69.4	35.0	42.3	85.8	35.9	13.8	46.7	-5.16
Combined removal	67.5	23.1	25.2	48.5	69.4	36.3	37.6	78.8	35.9	11.3	43.3	-8.55

is more notable, as it requires reconstructing hierarchical relations, a setting often associated with graph-based memory rather than flat textual stores. MEMLACE reconstructs the queried hierarchy from atomic relational memories, structured retrieval, and evidence-unit grouping, supplying the answer model with the locally relevant dependencies rather than a comprehensive hierarchical knowledge graph. Perfect accuracy on both families lifts the macro-average to 52.08%, and the gap over Mem-Agent is driven entirely by these two structured, evolving-state task types.

The counting and recommendation results identify a targeted extension opportunity for the framework. Neither method solves the count-based scenarios. In recommendation, MEMLACE answers 32.2% of the individual questions correctly, as computed from the question-level judge outputs, while reaching 8.33% at the scenario level under the benchmark’s 50% correctness threshold. These tasks depend on global numerical accumulation or ranking across many distributed preference signals, whereas MEMLACE is designed to preserve validity, provenance, and dependencies among evolving pieces of evidence. Dedicated aggregation operators, e.g., invoking a calculator or code-execution tool, could complement the evidence units produced by MEMLACE without requiring a more expensive global memory architecture.

VI. ABLATION STUDY

We conduct an ablation study to gain further insight into the importance and empirical impact of each of the MEMLACE components. We isolate the contribution of each component by evaluating variants of MEMLACE on the complete *BEAM* 100K subset using Qwen3.5-9B. Each run changes exactly one component, except the *combined removal* variant, which uses compact merge and disables both rerankers, batch evolution, temporal awareness, and lifecycle expansion. Table IV reports category-level results for every variant.

Lifecycle expansion and temporal awareness account for the two largest reductions, but their effects fall on distinct capabilities, which indicates that they address different failure modes. On the one hand, removing lifecycle expansion (−5.16) leaves active-memory retrieval intact but stops it from traversing supersession and contradiction links, and the loss concentrates on categories that require earlier and current

states together: knowledge update (51.9 $\rightarrow$ 35.0), multi-hop reasoning (55.3 $\rightarrow$ 42.3), and temporal reasoning (26.3 $\rightarrow$ 13.8). Contradiction resolution is unchanged at the reported precision (40.3 in both), indicating that the relevant conflicting memories were often retrieved independently as active anchors even without contradiction-link expansion. On the other hand, removing temporal awareness (−4.97) degrades contradiction resolution (40.3 $\rightarrow$ 26.6), preference following (84.4 $\rightarrow$ 75.6), and temporal reasoning (26.3 $\rightarrow$ 16.3). Temporal awareness here denotes MEMLACE’s use of evidence-grounded temporal metadata (τ_i), such as event times and validity periods, when consolidating and ordering memories, so that an ordered change over time is distinguished from a genuine conflict; without it, a valid update is more easily misread as a contradiction. The two components are therefore not interchangeable: connectivity determines which related memories are retrieved, whereas temporal grounding determines whether a difference between them is read as a genuine contradiction or an ordered change.

The remaining components lead to smaller accuracy reductions. The impacted categories clarify the contribution of each component. Removing batch evolution (−1.68) affects instruction following (72.5 $\rightarrow$ 60.6) and multi-hop reasoning (55.3 $\rightarrow$ 48.2) most, consistent with its role of recovering relations that span entries which were separately handled by the single-decision consolidation. Removing both rerankers (−1.23) produces the smallest performance drop, although instruction following decreases from 72.5 to 63.8, consistent with a component that reweights already-retrieved evidence rather than changing which evidence or relations exist. The merge comparison is the most diagnostic: the compact merge policy, which replaces the default linked-merge entries with a single synthesized memory, improves contradiction resolution (40.3 $\rightarrow$ 44.1) but substantially reduces multi-hop reasoning (55.3 $\rightarrow$ 43.0) for a net drop of 1.43 percentage points. Consolidating evidence into one entry thus helps when a single reconciled statement is sufficient, but discards the separate provenance and phrasing that multi-hop questions rely on. The default linked-merge policy applied in MEMLACE strikes a better trade-off.

The combined removal drops the overall score to 43.3%,

a larger reduction than any single ablation. We note that this combined reduction is smaller than the sum of the individual drops, indicating that the components overlap partially while still contributing separable effects. The degradation is concentrated on evolving and relation-dependent categories, i.e., contradiction resolution (23.1), knowledge update (36.3), multi-hop reasoning (37.6), and temporal reasoning (11.3), whereas event ordering (25.2) and summarization (35.9) remain close to the full configuration.

VII. CONCLUSION

We presented MEMORYLACE (MEMLACE), a long-term memory framework that represents the evolution of textual evidence through sparse lifecycle relations connecting repeated, superseded, and contradictory memories while preserving their original provenance. By retrieving and reasoning over relation-aware evidence units rather than isolated memories, MEMLACE enables language models to interpret current, historical, supporting, and conflicting evidence within a unified retrieval pipeline. Across BEAM and StructMemEval, MEMLACE achieves the strongest overall performance in same-backbone comparisons while substantially reducing runtime relative to Hindsight, the strongest reported reflective-memory baseline. Ablation studies further show that lifecycle expansion and temporal awareness account for most of these gains, providing empirical evidence that explicitly modeling the local lifecycle of textual evidence is sufficient to recover much of the reasoning benefit typically associated with substantially heavier memory architectures. These findings suggest that, in the evaluated settings, the principal challenge for long-term memory is not richer global structure but preserving how evidence evolves over time. Future work can therefore extend this lifecycle-aware interface with specialized operators for aggregation, counting, and numerical reasoning while retaining the same lightweight memory representation.

REFERENCES

- [1] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, et al., “Memory in the Age of AI Agents,” arXiv preprint arXiv:2512.13564, 2025.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” arXiv preprint arXiv:2304.03442, 2023.
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv preprint arXiv:2310.08560, 2023.
- [5] J. Liu, Y. Su, P. Xia, S. Han, Z. Zheng, C. Xie, M. Ding, and H. Yao, “SimpleMem: Efficient Lifelong Memory for LLM Agents,” arXiv preprint arXiv:2601.02553, 2026.
- [6] Z. Pan, Q. Wu, H. Jiang, X. Luo, H. Cheng, D. Li, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and J. Gao, “On Memory Construction and Retrieval for Personalized Conversational Agents,” arXiv preprint arXiv:2502.05589, 2025.
- [7] R. Salama, J. Cai, M. Yuan, A. Currey, M. Sunkara, Y. Zhang, and Y. Benajiba, “MemInsight: Autonomous Memory Augmentation for LLM Agents,” in *Proceedings of EMNLP*, 2025, pp. 33136–33152.
- [8] S. Zhou and J. Han, “A Simple Yet Strong Baseline for Long-Term Conversational Memory of LLM Agents,” arXiv preprint arXiv:2511.17208, 2025.
- [9] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic Memory for LLM Agents,” in *Advances in Neural Information Processing Systems*, 2025.
- [10] K. Li, X. Yu, Z. Ni, Y. Zeng, Y. Xu, Z. Zhang, X. Li, J. Sang, X. Duan, X. Wang, C. Liu, and J. Tan, “TiMem: Temporal-Hierarchical Memory Consolidation for Long-Horizon Conversational Agents,” arXiv preprint arXiv:2601.02845, 2026.
- [11] N. Zhang, X. Yang, Z. Tan, W. Deng, and W. Wang, “HiMem: Hierarchical Long-Term Memory for LLM Long-Horizon Agents,” arXiv preprint arXiv:2601.06377, 2026.
- [12] K. T.-i. Ong, N. Kim, M. Gwak, H. Chae, T. Kwon, Y. Jo, S.-w. Hwang, D. Lee, and J. Yeo, “Towards Lifelong Dialogue Agents via Timeline-Based Memory Management,” in *Proceedings of NAACL*, 2025, pp. 8631–8661.
- [13] M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell, “Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs,” arXiv preprint arXiv:2510.27246, 2025.
- [14] A. Shutova, A. Olenina, I. Vinogradov, and A. Sinitsin, “Evaluating Memory Structure in LLM Agents,” arXiv preprint arXiv:2602.11243, 2026.
- [15] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, “Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects,” arXiv preprint arXiv:2512.12818, 2025.
- [16] Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, et al., “In Prospect and Retrospect: Reflective Memory Management for Long-Term Personalized Dialogue Agents,” in *Proceedings of ACL*, 2025, pp. 8416–8439.
- [17] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory,” arXiv preprint arXiv:2504.19413, 2025.
- [18] Z. Li, C. Xi, C. Li, D. Chen, B. Chen, S. Song, S. Niu, H. Wang, J. Yang, C. Tang, et al., “MemOS: A Memory OS for AI System,” arXiv preprint arXiv:2507.03724, 2025.
- [19] A. Tekparmak and O. Kaya, “mem-agent: Equipping LLM Agents with Memory Using RL,” Hugging Face Community Article, Oct. 2025. [Online]. Available: <https://huggingface.co/blog/driaforall/mem-agent-blog>
- [20] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating Very Long-Term Conversational Memory of LLM Agents,” in *Proceedings of ACL*, 2024, pp. 13851–13870.
- [21] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “Long-MemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory,” in *Proceedings of ICLR*, 2025.
- [22] H. Tan, Z. Zhang, C. Ma, X. Chen, Q. Dai, and Z. Dong, “MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-Based Agents,” in *Findings of ACL*, 2025.
- [23] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi, et al., “Evo-Memory: Benchmarking LLM Agent Test-Time Learning with Self-Evolving Memory,” arXiv preprint arXiv:2511.20857, 2025.
- [24] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, “Zep: A Temporal Knowledge Graph Architecture for Agent Memory,” arXiv preprint arXiv:2501.13956, 2025.
- [25] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [26] Qwen Team, “Qwen3.5: Towards Native Multimodal Agents,” 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [27] OpenAI, “GPT-5.5 System Card,” 2026. [Online]. Available: <https://deploymentsafety.openai.com/gpt-5-5>
- [28] OpenAI, “GPT-4o mini: Advancing Cost-Efficient Intelligence,” 2024. [Online]. Available: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [29] Sentence-Transformers, “Cross-Encoder for MS MARCO: cross-encoder/ms-marco-MiniLM-L6-v2,” Hugging Face. [Online]. Available: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
- [30] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, et al., “Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models,” arXiv preprint arXiv:2506.05176, 2025.